

Physics of Diffusion Models in Generative AI

Reneiloe Nyathela

14 May 2026

Abstract

In this draft, I build a neural network (NN) from scratch using first principles. The NN predicts a Gaussian probability distribution given a set of initial inputs. I define the weights and biases, explore the choice of activation and loss functions, and implement the network using assumed real values. I walk through the forward pass and the training samples used, then interpret the output explaining what it means and why the network behaves as it does. Finally, I use the trained network to estimate probability distributions at earlier times from later time inputs, compare the reconstructed distributions against analytical solutions, and comment on the accuracy and limitations of the approach.

I am essentially teaching the NN the Green's function solution of the diffusion equation, this is the reduced Fokker-Planck equation, where the drift term is set to zero. It reduces to the diffusion equation:

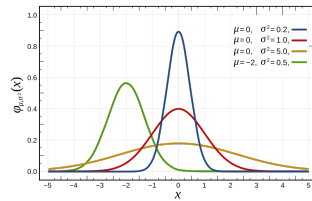
$$\frac{\partial P}{\partial t} = D \frac{\partial^2 P}{\partial x^2}$$

1 Learning Parameters

I define the probability distribution that takes in two input variables and spits out a single output. For this example, I am using the Gaussian probability distribution.

$$P(x, t) = \frac{1}{\sqrt{2\pi\sigma^2(t)}} e^{-\frac{(x-x_0)^2}{2\sigma^2(t)}} \quad (1)$$

Below is the Normal distribution at different time and different mean values.



Assuming the Gaussian is centered about $x=0$, and pure diffusion, thus,

$$\sigma(t)^2 = 2Dt$$

the distribution reduces to

$$P(x, t) = \frac{1}{\sqrt{4\pi Dt}} e^{-\frac{x^2}{4Dt}}$$

This is easy to work with and it assumes uniform distribution in all space. where:

- $P(x, t)$ is the learning function.

The NN takes in two variables as inputs, x and t . These inputs will be considered arbitrary for now. Let the two inputs be:

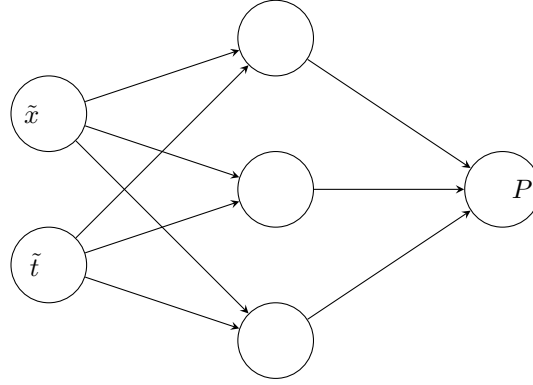
$$\vec{q}^{(0)} = \begin{pmatrix} x \\ t \end{pmatrix} \quad (2)$$

We know that $\vec{q}^{(0)}$ is raw data. I cannot directly feed x and t into the NN because they are units, so I define dimensionless variables $\tilde{x} = \frac{x}{L}$ and $\tilde{t} = \frac{t}{\tau}$. Or else the network will mix the quantities with incompatible units. Thus,

$$\vec{q}^{(0)} = \begin{pmatrix} \tilde{x} \\ \tilde{t} \end{pmatrix} \quad (3)$$

This is what I am going to feed into the first layer. The intuitive picture is like, I have two inputs, feed them into the hidden layer and then a output is generated.

Below is an example of the NN I will be using:



The first two input neurons describe the input variables x and t . The one output neuron at the end describes the prediction of the probability distribution. The middle three neurons, also known as the hidden layer or hidden neurons. This is where all the computations take place. I use this because it is much cleaner and simpler to work with.

1.1 Layer 1: Input $\rightarrow$ Hidden layer

Here I define the weights and the biases. Weights are numerical parameters linked to the inputs of a model, representing how much influence each input has on the final computation. In NN, they regulate the intensity of the connections between neurons, enabling the model to learn patterns and relationships between the provided data and the desired outputs. Biases, however, are independent constant values assigned to neurons rather than to particular inputs. They are added to the weighted sum before activation and act as an adjustment factor or shift. This allows neurons to produce meaningful outputs even when the combined weighted inputs are small, thereby improving the flexibility and learning capability of the network. Let:

$$W^{(1)} = \begin{pmatrix} W_{11}^1 & W_{12}^1 & W_{13}^1 \\ W_{21}^1 & W_{22}^1 & W_{23}^1 \end{pmatrix} \quad (4)$$

Weight $W^{(1)}$ is from input to hidden layer, hence the size of the matrix. From two neurons to three neurons. For the bias:

$$\vec{b}^{(1)} = \begin{pmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{pmatrix} \quad (5)$$

1.2 Layer 2: Hidden layer $\rightarrow$ Output

I have three neurons from the hidden layer, which will just spit out a single output value. Hence the weight and bias are define as follows:

$$W^{(2)} = \begin{pmatrix} W_1^2 \\ W_2^2 \\ W_3^2 \end{pmatrix} \quad (6)$$

Since there one output, bias defines a single scalar:

$$b^{(2)} = b^{(2)}$$

What I have defined above, so far, are all called the learning parameters.

	Weights	Biases
Layer 1	6	3
Layer 2	3	1

Table 1: Caption

In NNs, the learning parameters are the numbers the model learns from data during training. As I will later in the paper. All in all I have 13 parameters, the network will learn all of them.

[The superscripts in the learning parameters help remembering which layer I am dealing with.]

2 Forward Pass

In order to proceed I need the activation function, this is the rule that decides how much a neuron's signal should "turn on" or pass forward. It takes the weighted input plus bias and transforms it into the neuron's output, often adding nonlinearity so the network can learn more complex patterns. Below is the pre-activation function:

$$Z^{(1)} = (W^{(1)})^T \vec{q}^{(0)} + \vec{b}^{(1)} \quad (7)$$

When simplifying eq(7):

$$\Rightarrow Z^{(1)} = \begin{pmatrix} W_{11}^1 \tilde{x} + W_{21}^1 \tilde{t} + b_1^1 \\ W_{12}^1 \tilde{x} + W_{22}^1 \tilde{t} + b_2^1 \\ W_{13}^1 \tilde{x} + W_{23}^1 \tilde{t} + b_3^1 \end{pmatrix} = \begin{pmatrix} Z_1^1 \\ Z_2^1 \\ Z_3^1 \end{pmatrix} \quad (8)$$

Since the Gaussian outputs values in the range $P \in (0, \infty)$, always positive. It will be very useful that in the hidden layer I use the ReLU activation function, and to predict the final output of the NN I use the Softplus activation. I shall show their mechanisms.

2.1 ReLU

$$\vec{q}^{(1)} = ReLU(Z^{(1)}) = \begin{pmatrix} \max(0, Z_1^1) \\ \max(0, Z_2^1) \\ \max(0, Z_3^1) \end{pmatrix} \quad (9)$$

What the ReLU, hidden activation, does is it predicts the next input values from the initial ones I assumed. It uses the idea such that, if the value of any pre-activation is negative, set it to zero, else use its original value. This helps in avoiding obtaining negative values when predicting the Gaussian solution. The striking thing about it is its non linearity, it allow the NN to learn more complex patterns and become flexible. In the second layer, the pre-activation function:

$$Z^{(2)} = (W^{(2)})^T \vec{q}^{(1)} + \vec{b}^{(2)} \quad (10)$$

Simplifying eq(10):

$$Z^{(2)} = W_1^2 q_1 + W_2^1 q_2 + W_3^2 q_3 + b^{(2)} \quad (11)$$

2.2 Softplus

The Softplus activation function helps to predict the final Gaussian output. The predicted Gaussian probability function looks like this:

$$P'(x, t) = \text{Softplus}(Z^{(2)}) = \ln(1 + e^{Z^{(2)}}) \quad (12)$$

Simplifying and using all assumed variables, the predicted function looks like this:

$$P'(x, t) = \ln(1 + \exp((W^{(2)})^T((W^{(1)})^T \vec{q}^{(0)} + \vec{b}^{(1)}) + \vec{b}^{(2)}))$$

3 Loss function

Since this outputs a prediction, I need a single number that explicitly tells me how wrong my prediction is, compared to the true analytical solution. This is done using the loss function. The mean square error (MSE):

$$\mathcal{L} = \frac{1}{2}(P' - P)^2 \quad (13)$$

The $\frac{1}{2}$ will be convenient when taking derivatives later. To construct a flexible loss function, N number of training samples:

$$\mathcal{L} = \frac{1}{2N} \sum_{i=1}^N (P'_i - P_i)^2$$

where:

- if $\mathcal{L}$ is large - predictions are too far from the true value
- if $\mathcal{L}$ is too small - predictions are close to the true value
- if $\mathcal{L} = 0$ - perfect predictions

Obviously I need weights and biases that make the Loss as small as possible.

4 Backpropagation

This is the heart of training. I work backwards from the loss, I figure out how much weight and bias contributed the most to the error. Using chain rule:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial P'} \frac{\partial P'}{\partial Z^{(2)}} \frac{\partial Z^{(2)}}{\partial w} \quad (14)$$

Derivative of loss with respect to prediction

$$\frac{\partial \mathcal{L}}{\partial P'} = P' - P$$

Derivative of Softplus (Sigmoid of $Z^{(2)}$)

$$\frac{\partial P'}{\partial Z^{(2)}} = \sigma(Z^{(2)}) = \frac{e^{Z^{(2)}}}{1 + e^{Z^{(2)}}}$$

Output delta

$$\delta^2 = \frac{\partial \mathcal{L}}{\partial P'} \cdot \sigma(Z^{(2)}) = (P' - P) \cdot \frac{e^{Z^{(2)}}}{1 + e^{Z^{(2)}}}$$

Gradient with respect to $W^{(2)}$

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \vec{q}^{(1)} \cdot \delta^2$$

Gradient with respect to $b^{(2)}$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^2$$

Propagate delta backwards through $W^{(2)}$

$$\frac{\partial \mathcal{L}}{\partial \vec{q}^{(1)}} = W^{(2)} \cdot \delta^2$$

Applying ReLU derivative

$$\text{ReLU}'(Z) = \begin{cases} 1 & \text{if } Z > 0 \\ 0 & \text{if } Z \leq 0 \end{cases}$$

$$\delta^1 = \frac{\partial \mathcal{L}}{\partial \vec{q}^{(1)}} \cdot \text{ReLU}'(\mathbf{Z}^{(1)})$$

Gradient with respect to $W^{(1)}$

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \vec{q}^{(0)} \cdot (\delta^1)^T$$

Gradient with respect to $\vec{b}^{(1)}$

$$\frac{\partial \mathcal{L}}{\partial \vec{b}^{(1)}} = \delta^1$$

5 Gradient Descent

The Gradient descent helps nudge every weight and bias in the direction that reduces the loss. For this I use the derivatives computed. With the gradient descent, the NN moves against the gradients because the gradients points uphill and I desire going downhill towards lower loss, hence the the original parameter minus its derivative. This update is applied to every parameter, such that: For learning rate η , a small number controlling the step size.

$$W^{(2)} \leftarrow W^{(2)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(2)}}, \quad b^2 \leftarrow b^2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial b^2}$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(1)}}, \quad \vec{b}^{(1)} \leftarrow \vec{b}^{(1)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \vec{b}^{(1)}}$$

This completes the NN. I am going to insert a python code of this exact NN, assuming the inputs, the weights and the biases. I also assume the diffusion coefficient and the learning. What the NN does is the follow:

Forward pass $\longrightarrow$ Loss $\longrightarrow$ Backpropagation $\longrightarrow$ REPEATS

Below is the link to my github account, where I show the code build up from scratch. The files to view are NN(1).ipynb and NN(final).ipynb.

[Click here to view the files](#)